

Rockchip RK3506 Linux SDK 快速入门

文档标识: RK-JC-YF-A27

发布版本: V1.1.0

日期: 2024-11-28

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要描述了RK3506 Linux SDK的基本使用方法，旨在帮助开发者快速了解并使用RK3506 SDK开发包。

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

各芯片系统支持状态

芯片名称	Uboot版本	Kernel版本	Buildroot版本
RK3506	2017.9	6.1	2024.02

修订记录

日期	版本	作者	修改说明
2024-07-20	V0.0.1	Zack Huang	初始版本
2024-08-20	V0.1.0	Zack Huang	发布Beta版本
2024-09-20	V1.0.0	Zack Huang	发布Release版本
2024-11-28	V1.1.0	Zack Huang	增加部分内存配置说明

目录

Rockchip RK3506 Linux SDK 快速入门

1. SDK预编译镜像
2. 开发环境搭建
 - 2.1 准备开发环境
 - 2.2 安装库和工具集
 - 2.2.1 检查和升级主机的 `python` 版本
 - 2.2.2 检查和升级主机的 `make` 版本
 - 2.2.3 检查和升级主机的 `lz4` 版本
3. Docker环境搭建
4. 软件开发指南
 - 4.1 开发向导
 - 4.2 芯片资料
 - 4.3 Buildroot开发指南
 - 4.4 软件更新记录
5. 硬件开发指南
6. SDK 工程目录介绍
7. SDK交叉编译工具链介绍
 - 7.1 U-Boot 及Kernel编译工具链
 - 7.2 Buildroot工具链
 - 7.2.1 配置编译环境
 - 7.2.2 打包工具链
8. SDK编译说明
 - 8.1 SDK编译命令查看
 - 8.2 SDK板级配置
 - 8.3 SDK定制化配置
 - 8.4 全自动编译
 - 8.5 各模块编译
 - 8.5.1 U-Boot编译
 - 8.5.2 Kernel编译
 - 8.5.3 Recovery编译
 - 8.5.4 Buildroot编译
 - 8.5.4.1 Buildroot模块编译
 - 8.5.5 Yocto 编译
 - 8.6 固件的打包
9. 刷机说明
 - 9.1 Windows 刷机说明
 - 9.2 Linux 刷机说明
 - 9.3 系统分区说明
10. 产品化模块开发说明
 - 10.1 音频降噪算法调试介绍
 - 10.1.1 明确当前产品的类型和规格指标
 - 10.1.2 音频算法测试相关知识点介绍
 - 10.1.3 音频算法测试场景
 - 10.1.4 音频通路测试
 - 10.1.5 音频播放最大不失真信号测试
 - 10.1.6 音频扫频录放测试
 - 10.1.7 音频算法调试
 - 10.2 固件大小
 - 10.3 OTG-ID 和 OTG-VBUS-DET 配置说明
 - 10.3.1 OTG ID 配置
 - 10.4 视频软解
 - 10.5 MCU
 - 10.6 Flexbus(DVP、高速IO)

10.7 RS485硬件方案自动切换

10.8 实时性优化

10.9 EtherCat

10.10 CAN

10.11 DSMC

10.12 LVGL Demo

10.12.1 功能开启

10.13 显示部分

10.14 可读写文件系统

10.15 使用eMMC存储介质

1. SDK预编译镜像

使用RK3506 Linux SDK预编译镜像，开发人员可以省去从源代码编译整个操作系统的过程，直接将预编译的镜像刷入RK3506开发板，从而快速启动开发和进行相关评估、对比，可减少因编译问题导致的开发时间和成本浪费。

可从公开地址下载 [SDK固件下载点击这里](#)

固件路径：通用 **Linux SDK** 固件-> **Linux6.1 -> RK3506**

如果需要修改SDK代码或快速入门，请参考下面章节。

2. 开发环境搭建

2.1 准备开发环境

我们推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。其他的 Linux 版本可能需要对软件包做相应调整。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 22.04 或更高版本系统。

2.2 安装库和工具集

使用命令行进行设备开发时，可以通过以下步骤安装编译SDK需要的库和工具。

使用如下apt-get命令安装后续操作所需的库和工具：

```
sudo apt-get update && sudo apt-get install git ssh make gcc libssl-dev \
liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath \
diffstat binfmt-support qemu-user-static live-build bison flex fakeroot \
cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev \
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev \
libmpc-dev bc python-is-python3 python2
```

说明：

安装命令适用于Ubuntu22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。其中：

- python要求安装python 3.6及以上版本，此处以python 3.6为例。
- make要求安装 make 4.0及以上版本，此处以 make 4.2为例。
- lz4要求安装 lz4 1.7.3及以上版本。
- 编译Yocto需要VPN网络。

2.2.1 检查和升级主机的 `python` 版本

检查和升级主机的 `python` 版本方法如下：

- 检查主机 `python` 版本

```
$ python3 --version
Python 3.10.6
```

如果不满足 `python>=3.6` 版本的要求， 可通过如下方式升级：

- 升级 `python 3.6.15` 新版本

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

2.2.2 检查和升级主机的 `make` 版本

检查和升级主机的 `make` 版本方法如下：

- 检查主机 `make` 版本

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- 升级 `make 4.2` 新版本

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2.2.3 检查和升级主机的 lz4 版本

检查和升级主机的 lz4 版本方法如下：

- 检查主机 lz4 版本

```
$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.3, by Yann Collet ***
```

- 升级 lz4 新版本

```
git clone https://gitee.com/mirrors/LZ4_old1.git
cd LZ4_old1

make
sudo make install
sudo install -m 0755 lz4 /usr/bin/lz4
```

3. Docker环境搭建

为帮助开发者快速完成上面复杂的开发环境准备工作，我们也提供了交叉编译器Docker镜像，以便客户可以快速验证，从而缩短编译环境的构建时间。

使用Docker环境前，可参考如下文档进行操作

<SDK>/docs/cn/Linux/Docker/Rockchip_Developer_Guide_Linux_Docker_Deploy_CN.pdf。

已验证的系统如下：

发行版本	Docker 版本	镜像加载	固件编译
ubuntu 22.04	24.0.5	pass	pass
ubuntu 21.10	20.10.12	pass	pass
ubuntu 21.04	20.10.7	pass	pass
ubuntu 18.04	20.10.7	pass	pass
fedora35	20.10.12	pass	NR (not run)

Docker镜像可从网址 [Docker镜像](#) 获取。

4. 软件开发指南

4.1 开发向导

为帮助开发工程师更快上手熟悉 SDK 的开发调试工作，随 SDK 发布

《Rockchip_Developer_Guide_Linux_Software_CN.pdf》，可在 `docs/cn/RK3506` 目录下获取，并会不断完善更新

4.2 芯片资料

为帮助开发工程师更快上手熟悉 RK3506 的开发调试工作，随 SDK 发布《Rockchip RK3506 Datasheet V0.1-20240516.pdf》芯片手册。可在 `docs/cn/RK3506/Datasheet` 目录下获取，并会不断完善更新。

4.3 Buildroot开发指南

为帮助开发工程师更快上手熟悉Buildroot系统的开发调试，随 SDK 发布

《Rockchip_Developer_Guide_Buildroot_CN.pdf》开发指南，可在 `docs/cn/Linux/System` 目录下获取，并会不断完善更新。

4.4 软件更新记录

- 软件发布版本升级通过工程 xml 进行查看，具体方法如下：

```
.repo/manifests$ realpath rk3506_linux6.1_release.xml
# 例如:打印的版本号为v0.0.1，更新时间为20240720
# <SDK>/repo/manifests/release/rk3506_linux_alpha_v0.0.1_20240720.xml
```

- 软件发布版本升级更新内容通过工程文本可以查看，参考工程目录：

```
<SDK>/repo/manifests/RK3506_Linux6.1_SDK_Note.md
或者
<SDK>/docs/cn/RK3506/RK3506_Linux6.1_SDK_Note.md
```

- 板端可通过如下方式获取版本信息

```
buildroot:/# cat /etc/os-release
NAME=Buildroot
VERSION=linux-6.1-stan-rkr2
ID=buildroot
VERSION_ID=2024.02
PRETTY_NAME="Buildroot 2024.02"
OS="buildroot"
RK_BUILD_INFO="xxx Thu Apr 18 17:59:46 CST 2024 - rockchip_rk3506"
```

5. 硬件开发指南

硬件相关开发可以参考用户使用指南，在工程目录：

RK3506 硬件设计指南：

```
<SDK>/docs/cn/RK3506/Hardware/
```

6. SDK 工程目录介绍

SDK 工程目录包含有 buildroot、app、kernel、u-boot、device、docs、external、prebuilts等目录。采用 manifest来管理仓库，用repo工具来管理每个目录或其子目录会对应的 git 工程。

- app：存放上层应用 APP，主要是一些应用Demo。
- buildroot：基于 Buildroot（2024）开发的根文件系统。
- device/rockchip：存放芯片板级配置以及SDK编译和打包固件的脚本和文件等。
- docs：存放通用开发指导文档、芯片平台相关文档、Linux 系统开发相关文档、其他参考文档等。
- external：存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等。
- kernel：存放 Kernel 开发的代码。
- output：存放每次生成的固件信息、编译信息、XML、主机环境等。
- prebuilts：存放交叉编译工具链。
- rkbin：存放 Rockchip 相关二进制和工具。
- rockdev：存放编译输出固件,实际软链接到 `output/firmware`。
- tools：存放 Linux 和 Window 操作系统下常用工具。
- u-boot：存放基于 v2017.09 版本进行开发的 U-Boot 代码。
- rtos：存放rtos相关源码。
- yocto: 存放yocto相关编译脚本和代码。

7. SDK交叉编译工具链介绍

鉴于Rockchip Linux SDK目前只在Linux PC环境下编译，我们也仅提供Linux下的交叉编译工具链。prebuilt目录下的预置的工具链是给U-Boot和Kernel使用。具体Rootfs需要用各自对应的工具链，或者使用第三方工具链进行编译。

7.1 U-Boot 及Kernel编译工具链

SDK prebuilts目录预置交叉编译目前用于U-Boot 及Kernel编译，如下：

目录	说明
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	gcc arm 10.3.1 32位工具链

7.2 Buildroot工具链

7.2.1 配置编译环境

若需要编译单个模块或者第三方应用，需交叉编译环境进行配置。比如RK3506其交叉编译工具位于 `buildroot/output/rockchip_rk3506/host/usr` 目录下，需要将工具的 `bin/` 目录和 `arm-buildroot-linux-gnueabihf/bin/` 目录设为环境变量，在顶层目录执行自动配置环境变量的脚本：

```
source buildroot/envsetup.sh rockchip_rk3506
```

输入命令查看：

```
cd buildroot/output/rockchip_rk3506/host/usr/bin
./arm-linux-gcc --version
```

此时会打印如下信息：

```
arm-linux-gcc.br_real (Buildroot linux-6.1-stan-rkr3-40-gcec74d50ae-dirty) 12.3.0
```

7.2.2 打包工具链

Buildroot 支持将内置工具链打包为压缩包，以供第三方应用单独编译使用。有关如何打包工具链的详细信息，请参阅 Buildroot 官方文档：

```
buildroot/docs/manual/using-buildroot-toolchain.txt
```

在 SDK 中，可以直接运行以下命令来生成工具链包：

```
./build.sh bmake:sdk
```

生成的工具链包位于 `buildroot/output/*/images/` 目录，名为 `aarch64-buildroot-linux-gnu_sdk-buildroot.tar.gz`，供有需求的用户使用。解压后，`gcc` 的路径将是：

```
./aarch64-buildroot-linux-gnu_sdk-buildroot/bin/aarch64-buildroot-linux-gnu-gcc
```

8. SDK编译说明

SDK可通过 `make` 或 `./build.sh` 加目标参数进行相关功能的配置和编译。
具体参考 `device/rockchip/common/README.md` 编译说明。

为了确保SDK每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

一键编译

芯片	类型	参考机型	一键编译
RK3506	开发板	RK3506G EVB1	<code>./build.sh lunch:rockchip_rk3506_g_evb1_defconfig && ./build.sh</code>
RK3506	开发板	RK3506B EVB1	<code>./build.sh lunch:rockchip_rk3506_b_evb1_defconfig && ./build.sh</code>
RK3506	demo板	RK3506G DEMO	<code>./build.sh lunch:rockchip_rk3506_g_demo_defconfig && ./build.sh</code>

注意：

- **Rootfs**编译：
默认是编译Buildroot系统，如果需要其他系统，设置相应环境变量即可。比如编译buildroot系统: `RK_ROOTFS_SYSTEM=buildroot ./build.sh`
- **Kernel、U-boot**编译：其中需要制指定工具链。
- 比如SDK内置32位工具链：
`export CROSS_COMPILE=./prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-`

8.1 SDK编译命令查看

`make help` , 例如：

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconfig           - Same as oldconfig, but quietly, additionally update
deps
olddefconfig        - Same as synconfig but sets new symbols to their
default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

`make`实际运行是 `./build.sh`

即也可运行 `./build.sh <target>` 来编译相关功能，具体可通过 `./build.sh help` 查看具体编译命令。

```
./build.sh -h

##### Rockchip Linux SDK #####

Manifest: rk3506_linux6.1_beta_20240820_v0.1.0.xml

Log colors: message notice warning error fatal

sage: build.sh [OPTIONS]
Available options:
```

chip[:<chip>[:<config>]]	choose chip
defconfig[:<config>]	choose defconfig
*_defconfig	switch to specified defconfig
available defconfigs:	
rockchip_defconfig	
olddefconfig	resolve any unresolved symbols in .config
savedefconfig	save current config to defconfig
menuconfig	interactive curses-based configurator
config	modify SDK defconfig
print-parts	print partitions
list-parts	alias of print-parts
mod-parts	interactive partition table modify
edit-parts	edit raw partitions
new-parts:<offset>:<name>:<size>...	re-create partitions
insert-part:<idx>:<name>[:<size>]	insert partition
del-part:(<idx> <name>)	delete partition
move-part:(<idx> <name>):<idx>	move partition
rename-part:(<idx> <name>):<name>	rename partition
resize-part:(<idx> <name>):<size>	resize partition
misc	pack misc image
kernel-6.1[:dry-run]	build kernel 6.1
kernel[:dry-run]	build kernel
recovery-kernel[:dry-run]	build kernel for recovery
modules[:dry-run]	build kernel modules
linux-headers[:dry-run]	build linux-headers
kernel-config[:dry-run]	modify kernel defconfig
kconfig[:dry-run]	alias of kernel-config
kernel-make[:<arg1>:<arg2>]	run kernel make
kmake[:<arg1>:<arg2>]	alias of kernel-make
wifibt[:<dst dir>[:<chip>]]	build Wifi/BT
amp	build and pack amp system
buildroot-config[:<config>]	modify buildroot defconfig
bconfig[:<config>]	alias of buildroot-config
buildroot-make[:<arg1>:<arg2>]	run buildroot make
bmake[:<arg1>:<arg2>]	alias of buildroot-make
rootfs[:<rootfs type>]	build default rootfs
buildroot	build buildroot rootfs
yocto	build yocto rootfs
debian	build debian rootfs
recovery	build recovery
security-createkeys	create keys for security
security-misc	build misc with system encryption key
security-ramboot	build security ramboot
security-system	build security system
loader[:dry-run]	build loader (u-boot)
uboot[:dry-run]	build u-boot
u-boot[:dry-run]	alias of uboot
uefi[:dry-run]	build uefi
extra-parts	pack extra partition images
firmware	pack and check firmwares
edit-package-file	edit package-file
edit-ota-package-file	edit package-file for OTA
updateimg	build update image
ota-updateimg	build update image for OTA
all	build images

release	release images and build info
all-release	build and release images
shell	setup a shell for developing
cleanall	cleanup
clean[:module[:module]]...	cleanup modules
available modules:	
all	
amp	
config	
extra-parts	
firmware	
kernel	
loader	
misc	
recovery	
rootfs	
security	
updateimg	
post-rootfs <rootfs dir>	trigger post-rootfs hook scripts
help	usage
Default option is 'all'.	

8.2 SDK板级配置

进入工程 `<SDK>/device/rockchip/rk3506` 目录：

板级配置	说明
rockchip_defconfig	默认配置, 具体会软链接到RK3506G EVB1开发板配置
rockchip_rk3506_g_evb1_defconfig	适用于使用Linux方案的RK3506G EVB1 板子
rockchip_rk3506_b_evb1_defconfig	适用于使用Linux方案的RK3506B EVB1 板子
rockchip_rk3506_g_demo_defconfig	适用于使用Linux方案的RK3506G DEMO 板子
rockchip_rk3506_g_evb1_amp_defconfig	适用于使用AMP方案的RK3506G EVB1 板子 CPU0~CPU1: Linux CPU2: RT_Thread
rockchip_rk3506_g_evb1_smp_defconfig	适用于使用RTOS SMP方案的RK3506G EVB1 板子 CPU0~CPU2: RT_Thread

注意：使用RTOS SMP和AMP方案请查阅文档：

SDK\docs\cn\Common\AMP\Rockchip_Developer_Guide_AMP_CN.pdf

可通过 `make lunch` 或者 `./build.sh lunch` 进行配置

```
$ ./build.sh lunch

You're building on Linux
Lunch menu...pick a combo:

1. rockchip_rk3506_b_evbl_defconfig
2. rockchip_rk3506_g_demo_defconfig
3. rockchip_rk3506_g_evbl_defconfig
...
Which would you like? [1]:
```

其他功能的配置可通过 `make menuconfig` 来配置相关属性。

8.3 SDK定制化配置

SDK可通过 `make menuconfig` 进行相关配置，目前可配组件主要如下：

```
(rockchip_rk3506_g_evbl_defconfig) Name of defconfig to save
[*] Rootfs (Buildroot|Debian|Yocto) --->
[*] Loader (U-Boot) --->
[ ] AMP (Asymmetric Multi-Processing System)
[*] Kernel (Embedded in an Android-style boot image) --->
    Boot (Android-style boot image) --->
[*] Recovery (based on Buildroot) --->
    *** Security feature depends on buildroot rootfs ***
    Extra partitions (oem, userdata, etc.) --->
    Firmware (partition table, misc image, etc.) --->
[*] Update (Rockchip update image) --->
    Others configurations --->
```

- **Rootfs:** 这里的Rootfs代表“根文件系统”，在这里可选择Buildroot、Yocto、Debian等不同的根文件系统配置。
- **Loader (u-boot):** 这是引导加载器的配置，通常是u-boot，用于初始化硬件并加载主操作系统。
- **AMP:** 多核异构启动方案，适用于需要实时性能的应用场景。
- **Kernel:** 这里配置内核选项，定制适合自己的硬件和应用需求的Linux内核。
- **Boot:** 这里配置Boot分区支持格式。
- **Recovery (based on buildroot):** 这是基于buildroot的recovery环境的配置，用于系统恢复和升级。
- **PCBA test (based on buildroot):** 这是一个基于buildroot的PCBA（印刷电路板组装）测试环境的配置。
- **Security:** 安全功能开启，包含Secureboot开启方式、Optee存储方式、烧写Key等。
- **Extra partitions:** 用于配置额外的分区。
- **Firmware:** 在这里配置固件相关选项。
- **Update (Rockchip update image):** 用于配置Rockchip完整固件的选项。
- **Others configurations:** 其他额外的配置选项。

通过 `make menuconfig` 配置界面提供了一个基于文本的用户界面来选择和配置各种选项。

配置完成后，使用 `make savedefconfig` 命令保存这些配置，这样定制化编译就会根据这些设置来进行。

通过以上config，可选择不同Rootfs/Loader/Kernel等配置，进行各种定制化编译，这样就可以灵活地选择和配置系统组件以满足特定的需求。

8.4 全自动编译

为了确保软件开发套件（SDK）的每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

进入工程根目录执行以下命令自动完成所有的编译：

```
./build.sh all # 只编译模块代码（u-Boot, kernel, Rootfs, Recovery）
               # 需要再执行`./build.sh ./mkfirmware.sh`进行固件打包

./build.sh     # 编译模块代码（u-Boot, kernel, Rootfs, Recovery）
               # 打包成update.img完整升级包
               # 所有编译信息复制和生成到out目录下
```

默认是 Buildroot，可以通过设置环境变量 `RK_ROOTFS_SYSTEM` 指定不同 rootfs。`RK_ROOTFS_SYSTEM` 目前只可设定一种系统：buildroot。

比如需要 debain 可以通过以下命令进行生成：

```
export RK_ROOTFS_SYSTEM=buildroot
./build.sh
或
RK_ROOTFS_SYSTEM=debian ./build.sh
```

8.5 各模块编译

8.5.1 U-Boot编译

```
./build.sh uboot
```

针对RK3506平台，SDK中对uboot有相关裁剪定制的配置，如下说明：

配置名称	说明
rk3506_defconfig	RK3506基础配置
rk3506_tb.config	RK3506 快速启动子配置
rk-amp.config	RK3506 AMP子配置

使用方法：基础配置+ 子配置

以RK3506G EVB1 开发板为例：使用快速启动方案，配置文件
SDK/device/rockchip/rk3506/rockchip_rk3506_g_evb1_defconfig如下：

```
RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
```

```

RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y

```

如果不使用快起方案，配置文件SDK/device/rockchip/rk3506/rockchip_rk3506_g_evb1_defconfig如下：

```

RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y

```

主要差异：

```

RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"  #这个为使用快起的uboot fragments配置
RK_UBOOT_CFG_FRAGMENTS="rk3506"    #这个为正常使用uboot的配置

```

8.5.2 Kernel编译

- 方法一

```
./build.sh kernel
```

- 方法二


```
cd kernel
export CROSS_COMPILE=./prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-
make ARCH=arm rk3506_defconfig;
make ARCH=arm rk3506g-evb1-v10.img -j24
```

注意：`rk3506g-evb1-v10`可替换具体的硬件dts

针对RK3506平台，SDK中对kernel有相关裁剪定制的配置，如下说明：

配置名称	说明
rk3506_defconfig	RK3506基础配置
rk3506-display.config	RK3506显示子配置
rk3506-wifibt.config	RK3506Wi-Fi/BT 子配置
rk3506_usbhost.config	RK3506 USB HOST 子配置
rk3506-usb-otg.config	RK3506 USB OTG 子配置
rk3506-usb-peripheral.config	RK3506 USB PERIPHERAL 子配置
rk3506-ethercat.config	RK3506 ETHERCAT 子配置

使用方法：基础配置+ 子配置

以RK3506G EVB1 开发板为例：配置文件SDK/device/rockchip/rk3506/rockchip_rk3506_g_evb1_defconfig如下：

```
RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506g-evb1-v10"
RK_BOOT_COMPRESSED=y
RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
RK_FLASH_SIZE=2048
RK_EXTRA_PARTITION_1_FSTYPE="ubi"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
RK_EXTRA_PARTITION_2_FSTYPE="ubi"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y
```

8.5.3 Recovery编译

进入工程根目录执行以下命令自动完成 Recovery 的编译及打包。

```
<SDK># ./build.sh recovery
```

编译后在 Buildroot 目录 `output/rockchip_rk3506_recovery/images` 生成 `recovery.img`。

注：recovery.img 是包含内核，所以每次 Kernel 更改，Recovery 是需要重新打包生成。Recovery重新打包的方法如下：

```
<SDK># source buildroot/envsetup.sh
<SDK># cd buildroot
<SDK># make recovery-reconfigure
<SDK># cd -
<SDK># ./build.sh recovery
```

注：Recovery是非必需的功能，有些板级配置不会设置

8.5.4 Buildroot编译

针对RK3506平台，SDK中对buildroot有相关裁剪定制的配置，如下说明：

配置名称	说明
rockchip_rk3506_defconfig	RK3506默认的配置
rockchip_rk3506_tiny_defconfig	RK3506裁剪掉显示部分的配置，用于一些没有带显示屏幕的产品

使用方法：在device/rockchip/.chip/rk3506/rockchip_rk3506_XXXX_defconfig中修改：

```
diff --git a/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
b/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
index bc06b077..887c9e24 100644
--- a/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
+++ b/.chips/rk3506/rockchip_rk3506_g_evbl_defconfig
@@ -1,5 +1,6 @@
RK_ROOTFS_UBI=y
RK_WIFIBT_CHIP="AP6256"
+RK_BUILDROOT_CFG="rk3506_tiny"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG_FRAGMENTS="rk3506_tb"
RK_UBOOT_SPL=y
```

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh rootfs
```

编译后在Buildroot目录 `output/rockchip_rk3506/images` 下生成不同格式的镜像, 默认使用rootfs.ubi格式。

具体可参考Buildroot开发文档参考:

`<SDK>/docs/cn/Linux/System/Rockchip_Developer_Guide_Buildroot_CN.pdf`

8.5.4.1 Buildroot模块编译

可通过 `source buildroot/envsetup.sh` 来设置不同芯片和目标功能的配置

```
$ source buildroot/envsetup.sh
Top of tree: rk3506

You're building on Linux
Lunch menu...pick a combo:

49. rockchip_rk3506
50. rockchip_rk3506_recovery
...

Which would you like? [1]:
```

默认选择 `rockchip_rk3506`。然后进入RK3506的Buildroot目录, 开始相关模块的编译。

其中 `rockchip_rk3506_recovery` 是用来编译Recovery模块。

比如编译 `rockchip-test` 模块, 常用相关编译命令如下:

进入 `buildroot` 目录

```
<SDK># cd buildroot
```

- 删除并重新编译 `rockchip-test`

```
buildroot# make rockchip-test-recreate
```

- 重编 `rockchip-test`

```
buildroot# make rockchip-test-rebuild
```

- 删除 `rockchip-test`

```
buildroot# make rockchip-test-dirclean
```

或者

```
buildroot# rm -rf output/rockchip_rk3506/build/rockchip-test-master/
```

8.5.5 Yocto 编译

进入工程目录根目录执行以下命令自动完成 Rootfs 的编译及打包：

```
./build.sh yocto
```

编译后在 yocto 目录 build/lastest 下生成 rootfs.img。

默认用户名登录是 root。Yocto 更多信息请参考 [Rockchip Wiki](#)。

FAQ:

- 上面编译如果遇到如下问题情况：

```
Please use a locale setting which supports UTF-8 (such as LANG=en_US.UTF-8).  
Python can't change the filesystem locale after loading so we need a UTF-8  
when Python starts or things won't work.
```

解决方法:

```
locale-gen en_US.UTF-8  
export LANG=en_US.UTF-8 LANGUAGE=en_US:en LC_ALL=en_US.UTF-8
```

或者参考 [setup-locale-python3](#)

- 如果遇到git权限问题，导致编译出错。

由于git新版本增加了CVE-2022-39253安全检测补丁，而如果旧版本的Yocto就需要poky包含如下才可修复：

```
commit ac3eb2418aa91e85c39560913c1ddfd2134555ba  
Author: Robert Yang <liezhi.yang@windriver.com>  
Date:   Fri Mar 24 00:09:02 2023 -0700  
  
    bitbake: fetch/git: Fix local clone url to make it work with repo  
  
    The "git clone /path/to/git/objects_symlink" couldn't work after the  
    following  
    change:  
  
    https://github.com/git/git/commit/6f054f9fb3a501c35b55c65e547a244f14c38d56
```

或者通过回退PC的git版本到V2.38或早期版本也行， 比如下：

```
$ sudo apt update && sudo apt install -y libcurl4-gnutls-dev  
  
git clone https://gitee.com/mirrors/git.git --depth 1 -b v2.38.0  
cd git  
make git -j8  
make install  
sudo install -m 0755 git /usr/bin/git
```

8.6 固件的打包

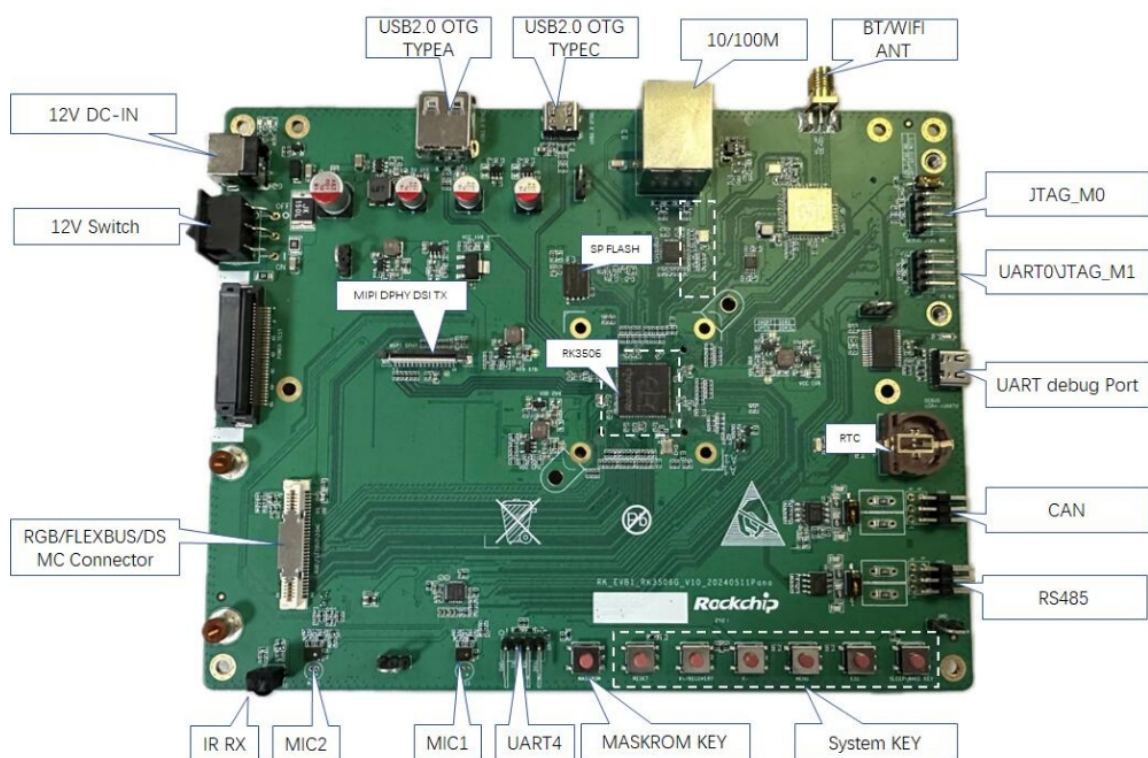
上面 Kernel/U-Boot/Recovery/Rootfs 各个部分的编译后，进入工程目录根目录执行以下命令自动完成所有固件打包到 `output/firmware` 目录下：

固件生成：

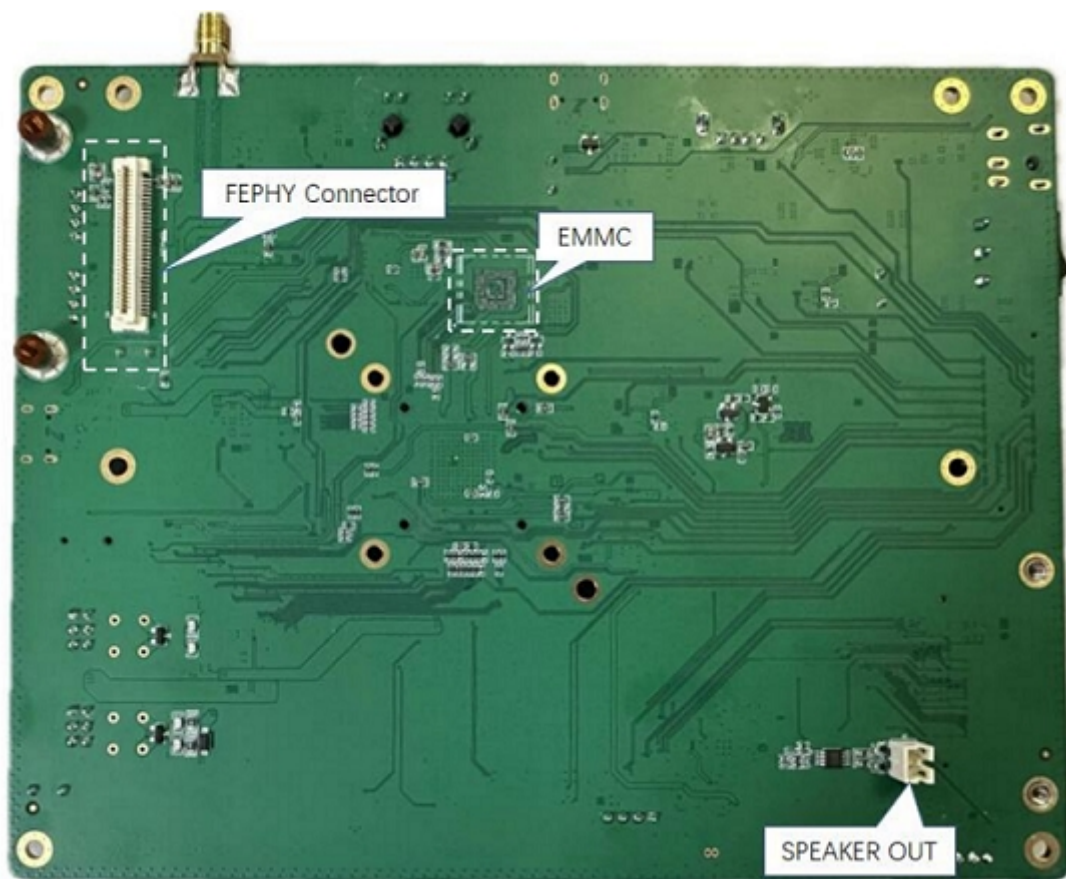
```
./build.sh firmware
```

9. 刷机说明

RK3506 EVB 1开发板正面接口分布图如下：



RK3506 EVB1 开发板背面接口分布图如下：



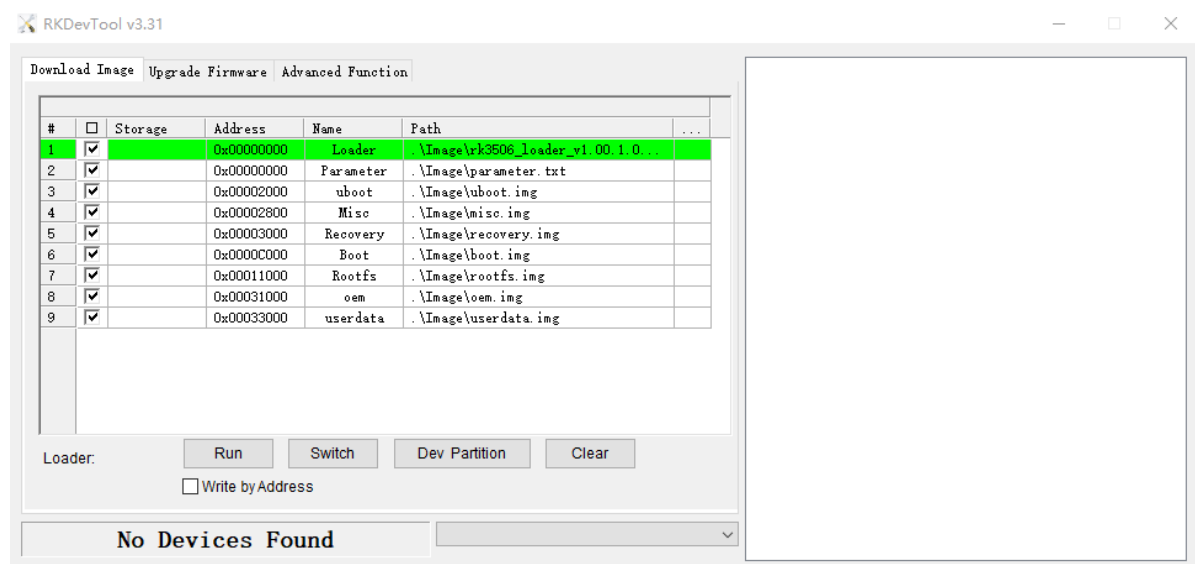
9.1 Windows 刷机说明

SDK 提供 Windows 烧写工具(工具版本需要 V3.28 或以上), 工具位于工程根目录:

```
tools/
└─ windows/RKDevTool
```

如下图, 编译生成相应的固件后, 设备烧写需要进入 MASKROM 或 BootROM 烧写模式, 连接好 USB 下载线后, 按住按键“MASKROM”不放并按下复位键“RST”后松手, 就能进入MASKROM 模式, 加载编译生成固件的相应路径后, 点击“执行”进行烧写, 也可以按 “recovery”按键不放并按下复位键 “RST” 后松手进入 loader 模式进行烧写, 下面是 MASKROM 模式的分区偏移及烧写文件。

(注意: Windows PC 需要在管理员权限运行工具才可执行)



注：烧写前，需安装最新 USB 驱动，驱动详见：

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

9.2 Linux 刷机说明

Linux 下的烧写工具位于 tools/linux 目录下(Linux_Upgrade_Tool 工具版本需要 V2.36 或以上)，请确认你的板子连接到 MASKROM/loader rockusb。比如编译生成的固件在 rockdev 目录下，升级命令如下：

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -misc rockdev/misc.img
sudo ./upgrade_tool di -b rockdev/boot.img
sudo ./upgrade_tool di -recovery rockdev/recovery.img
sudo ./upgrade_tool di -oem rockdev/oem.img
sudo ./upgrade_tool di -rootfs rockdev/rootfs.img
sudo ./upgrade_tool di -userdata rockdev/userdata.img
sudo ./upgrade_tool rd
```

或升级打包后的完整固件：

```
sudo ./upgrade_tool uf rockdev/update.img
```

或在根目录，机器在 MASKROM 状态运行如下升级：

```
./rkflash.sh
```

9.3 系统分区说明

默认分区说明（下面是 RK3506 EVB 分区参考）

- uboot 分区：供 uboot 编译出来的 uboot.img。
- misc 分区：供 misc.img，给 recovery 使用。
- boot 分区：供 kernel 编译出来的 boot.img。
- recovery 分区：供 recovery 编译出的 recovery.img。
- backup 分区：预留，暂时没有用。
- rootfs 分区：供 buildroot、debian 或 yocto 编出来的 rootfs.img。
- oem 分区：给厂家使用，存放厂家的 APP 或数据。挂载在 /oem 目录。
- userdata 分区：供 APP 临时生成文件或给最终用户使用，挂载在 /userdata 目录下。

10. 产品化模块开发说明

10.1 音频降噪算法调试介绍

在面临音频调参需求时，大部分开发人员都习惯对着音频算法接口和参数盲调一通，而忽视了前端输入的信号是否是正确的、不失真的。其实，算法调试仅仅是在整个音频效果通路的末端起到辅助作用，真正需要重视的，往往是算法前端的声学结构、以及硬件（元器件）指标等环节。只有算法前端获取的信号不失真，算法才能真正有效地发挥它的作用。

因此，当前的产品类型和规格指标，可以在调参之前先明确好，以取得事半功倍的效果。

10.1.1 明确当前产品的类型和规格指标

首先，开发人员要了解当前的产品类型以及对音频效果的需求。比如：使用几个麦克风，几个喇叭播放？产品类型是IPC，还是会议拾音系统等？是否有双向通话需求？通常IPC类产品不会需要超过2个麦克风拾音，而会议系统要求的麦克风数量较多，若是多麦克风场景下的麦克风阵列类型（如4MIC的拓扑，是环麦还是线麦）。

其次，确定好产品设计要求的拾音距离范围（如3米到10米），以及播放响度，以下信息，可通过器件手册查询：

- 麦克风是模拟麦（驻极体居多）还是数字麦？通常模拟麦对远场拾音会有更好的效果
- 麦克风的SNR（通常不低于65dB），灵敏度，以模拟麦为例，5米以上远场拾音，不低于-32dBV
- 喇叭播放输出的功率和负载，如2W@8ohm，
- 产品结构设计相关图片信息，如MIC孔径，还有和SPK的位置、距离、分布等
- 使用硬件回采还是软件回采。目前以软件回采应用居多

接着，还需要确定产品使用的录放的采样率，比如通常为16kHz或者8kHz。若需要使用AI降噪，或者哭声检测的场景，需要使用16kHz及以上的采样率。

10.1.2 音频算法测试相关知识点介绍

用户大多数需要开启音频算法，相关的模块主要有ANR，用于消除环境噪声；还有在通话场景下开启回声消除AEC（Acoustic Echo Cancellation），用于消除通话对讲时远端用户传来的回声。以下框图描述了经典回声消除算法在工作时的数据流向和处理效果：

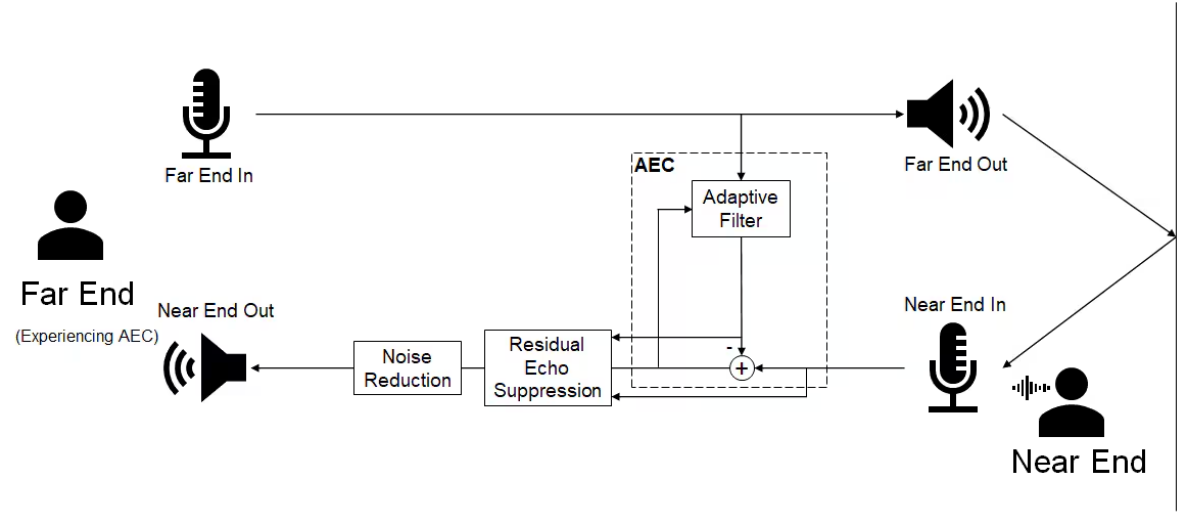


图 回声消除工作流

其中，有几个名词术语需要明确并统一：

- 近端（Near End）：以待测机器的角度看，比如主讲人对着机器的MIC说话，我们称之为“近端输入的人声”，接着，这说话声，通过网络传输到远端设备播放出来。
- 远端（Far End）：指与待测机器通讯的远端设备，比如测试人员对着远端设备的MIC说话，通过网络传输到待测机器播放出来。同时，这部分数据也通过回采（不论是软件回采还是硬件回采）的方式，存储在待测机器的另一个录音通道里，作为回声消除的参考信号。

10.1.3 音频算法测试场景

用户通话对讲时通常会遇到以下使用场景：

- 近端不说话，远端说话

在这种情况下，系统处于远端单讲状态，远端声音通过近端扬声器播放，再被麦克风采集回去（麦克风输入包含回声和噪声）。这个通常是音频通路测试完以后，音频效果调参最先测试及关注的。测试人员可以在独立的空间内，尽可能使用自然人声向远端设备（如手机或另一台对讲设备）说话，然后用远端设备听是否有自己的说话声。如果没有，说明远端单讲下回声抑制的效果较为理想；反之，会有残留回声从设备再回传回来，需要进一步调参优化。

- 近端说话，远端不说话

在这种情况下，系统处于近端单讲状态，远端没有说话（本机扬声器没有声音），那么也就不存在回声，此时实际相当于麦克风采集到的只有近端主讲人声和环境声。

- 近端和远端同时说话

在这种情况下，系统处于双讲状态，该状态下自适应滤波器容易发散，导致回声消除力度减小。因此，这时候更需要关注下播放音量是否过大，过大的回声能量导致重新被近端麦克风采集导致截幅失真等。

因此，在测试回声消除效果的时候，需要以上三个情况依次测试并关注每轮测试的结果。如果仅需要做降噪算法，可以只关注近端说话，远端不说话这个测试场景。

10.1.4 音频通路测试

在算法调试之前，我们需要对音频通路做测试，调整合适的录放增益，将最佳的电信号数据提供给算法，才可能获得更理想的处理效果。

10.1.5 音频播放最大不失真信号测试

如果一款产品有通话对讲的需求，我们应该先明确喇叭的播放功率和负载，并做好播放最大不失真信号的增益标定。否则，喇叭的失真会引起严重的漏回声异常。

所以，我们可以播放一个0dBFS 1kHz正弦波，然后通过示波器测量功率放大器（PA）输出或者喇叭两端的电压幅值，以确定实际输出的最大不失真电压是否满足产品定位要求。但是，由于PA通常是用ClassD驱动放大，直接测量PA输出或者喇叭两端，会包含几百kHz的高频载波信号。因此，如下图所示，我们可以在喇叭两端分别串上RC电阻形成一阶滤波电路，阻容配比可以为500ohm/10nF，截至频率约为32kHz，既可以滤除高频载波信号，也较为接近人耳可听频率范围上限20kHz。

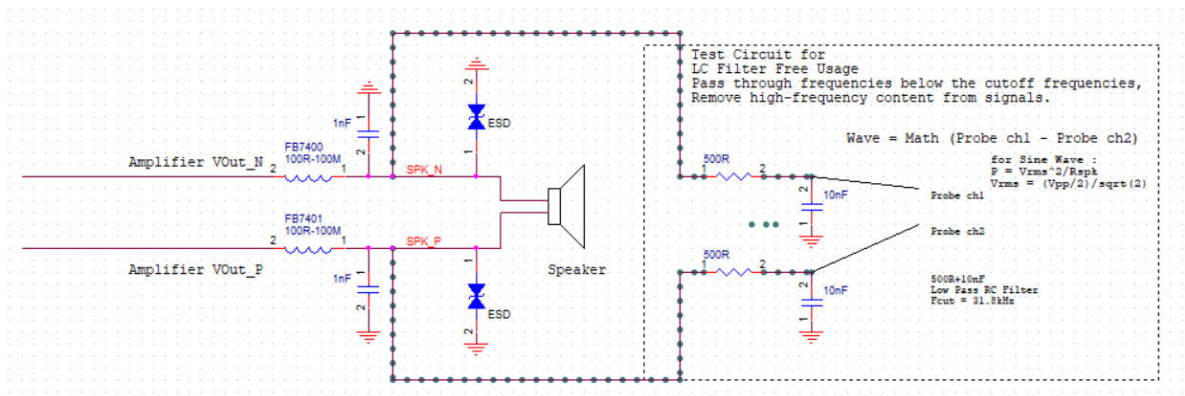


图 测量喇叭输出幅度和添加RC滤波电路的方法

播放0dBFS 1kHz正弦波可以使用如下命令，注意播放的是原始数据，不能开启AO VQE算法：

```
rk_mpi_ao_test -i /data/sine1000_16khz_2ch.wav --sound_card_name=hw:0,0 --
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

如果有tinyalsa或者alsa-utils，也可以使用如下命令播放测试：

```
tinyplay /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -b 2 -D 0
```

```
aplay -Dhw:0,0 /data/sine1000_16khz_2ch.wav -r 16000 -c 2 -f S16_LE
```

经过一番播放增益调整，我们预期能在喇叭端测量到如下信号：



图 实测RC滤波后0dBFS 1kHz正弦波最大不失真电压

其中黄色和绿色的波形分别为喇叭的P端和N端，粉色的是示波器经过math sub运算获得的差分电压幅度。如果一个产品设计喇叭的额定功率最高为1.5W@8ohm，那么从粉色的峰-峰值电压9.8V可以得到：

$$P = V_{rms}^2 / R = ((V_{pp}/2)/\sqrt{2})^2 / R = ((9.8/2)/\sqrt{2})^2 / 8 \approx 1.5W$$

实测功率数值上刚好满足设计要求的1.5W。

10.1.6 音频扫频录放测试

首先第一步，先配置好回采通路。如果使用Rockit框架。因为rk_mpi_ai_test默认输出的/tmp/cap_out.pcm有1MB大小的限制，如果需要长时间录音，可以执行以下命令，将原始音频数据导出为/tmp/dbg_capin.pcm：

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm
```

然后执行不开VQE算法的播放命令：

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --  
out_rate 16000 --out_ch 2 --output /tmp
```

同时，还需要本机播放一个0dBFS的扫频文件：

```
rk_mpi_ao_test -i /data/sweep_16k_20_8000_16b_2ch.wav --sound_card_name=hw:0,0 --  
device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

这样，可以获算法处理前的原始录音数据：

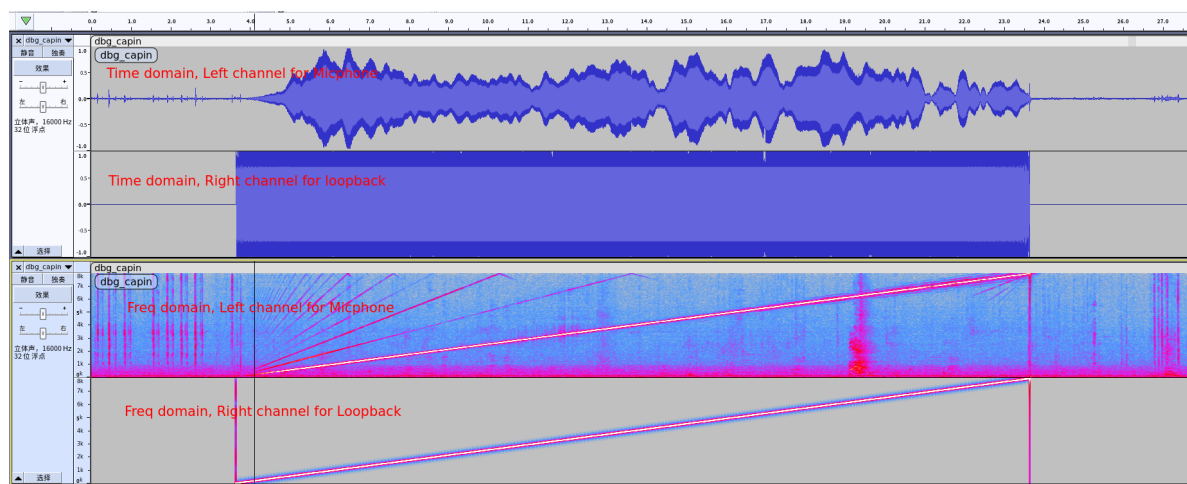


图 自录自播扫频测试

如上图可以看到，录到的音频数据是2ch的，并分别显示了2ch的时域和频域的状态。左声道为麦克风实时采集的数据，右声道为回采到刚才播放时的音频数据，且数字回采的PCM和播放出去的PCM内容是一致的。

如果喇叭增益太大，播放0dBFS扫频时可能会测到MIC录到的数据会有截幅异常：

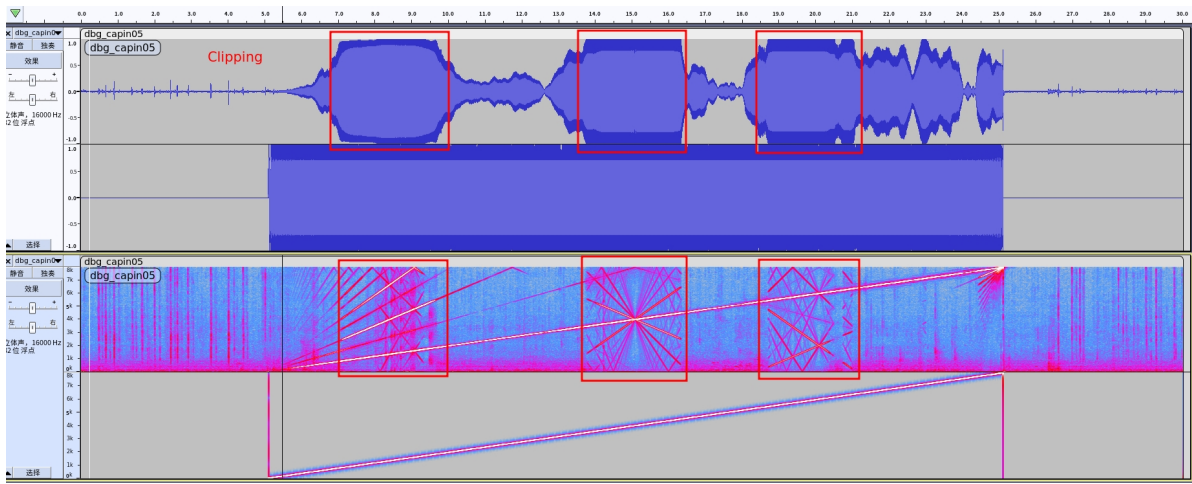


图 10-17 自录自播扫频测试截幅异常

可以将时间轴放大，看到上图红框的部分有明显的截幅失真。失真的信号对于后端算法的输入来说是不可逆的异常，会影响算法处理的结果。

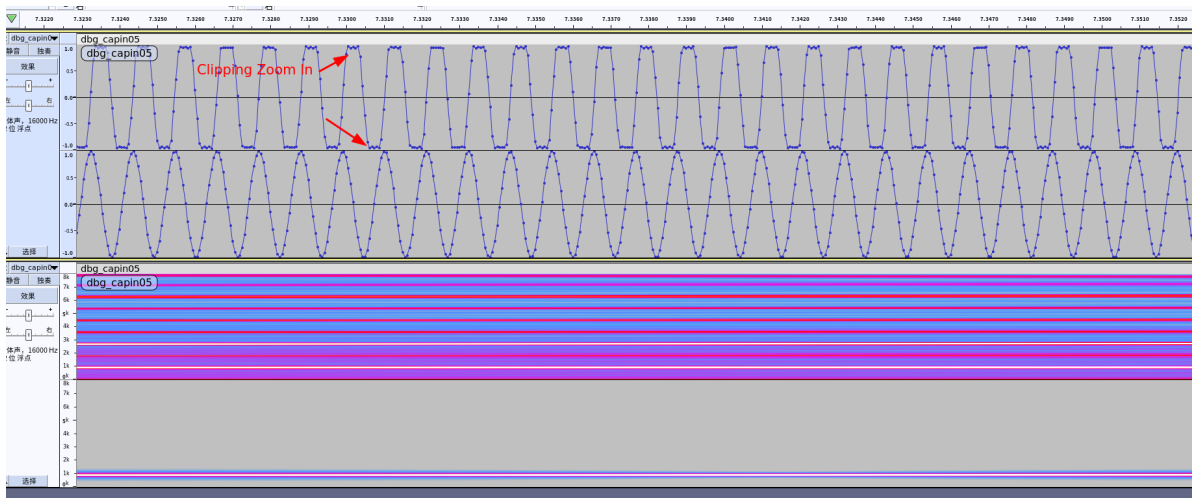


图 10-18 自录自播扫频测试截幅异常示意图

因此，如果有在此情况下遇到录音截幅失真，建议先调小前端的codec模拟增益。

10.1.7 音频算法调试

基于上面的音频通路测试，我们理应获得了较理想的输入信号数据，在应用集成算法之前，我们可以先确认一下测试板中是否包含了算法库和AI VQE的JSON配置参数，释放的SDK中它们默认路径下的位置分别是：

```
/oem/usr/lib/librkaudio_vqe.so
/oem/usr/lib/librkaudio_common.so
/oem/usr/share/vqefiles/config_aivqe.json
```

然后，在应用执行之前，我们先配置一些调试相关的环境变量，以生成我们需要的信息：

```
export rt_debug_cap_file=/tmp/dbg_capin.pcm // 兼容旧版本rockit，导出算法处理前的录音数据
export rt_debug_skv_out_file=/tmp/dbg_skvout.pcm // 兼容旧版本rockit，导出AI VQE算法处理后的录音数据
export rt_debug_skv_ao_out_file=/tmp/dbg_ao_skvout.pcm // 兼容旧版本rockit，导出AO VQE算法处理后的录音数据
export rt_debug_playback_file=/tmp/dbg_playback_out.pcm // 兼容旧版本rockit，导出所有AO算法处理后的录音数据，应和数字回采信号相同
export rt_dump_skv_param=true // rockit版本通用，打印AI VQE算法运行时的参数配置
export RKAUDIO_SKV_DUMP_CONF=true // 新版本算法库使用，打印AI VQE算法运行时的参数配置
export rt_dump_skv_inout=true // 新版本rockit使用，导出AI VQE算法处理前后的录音数据，默认生成至/tmp目录下。如：/tmp/dbg-skv-in-2ch-16000.pcm和/tmp/dbg-skv-out-1ch-16000.pcm
export rt_dump_skv_ao_inout=true // 新版本rockit使用，导出AO VQE算法处理前后的录音数据，默认生成至/tmp目录下。如：/tmp/dbg-skv-ao-in-2ch-16000.pcm和/tmp/dbg-skv-ao-out-1ch-16000.pcm
```

可以使用如下命令开启AI VQE算法，且输出为1ch，所以这里'--out_ch 1':

```
rk_mpi_ai_test --sound_card_name=hw:0,0 --device_rate 16000 --device_ch 2 --out_rate 16000 --out_ch 1 --output /tmp --vqe_enable 1 --vqe_cfg /oem/usr/share/vqefiles/config_aivqe.json
```

同时，可以播放本地存好的远端人声音频：

```
rk_mpi_ao_test -i /userdata/remote_voice.wav --sound_card_name=hw:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2
```

如果播放也需要走VQE算法的话，也可以加上使能AO VQE的配置参数：

```
rk_mpi_ao_test -i /userdata/remote_voice.wav --sound_card_name=hw:0,0 --device_ch=2 --device_rate=16000 --input_rate=16000 --input_ch=2 --vqe_cfg /oem/usr/share/vqefiles/config_aovqe.json --vqe_enable=1
```

这样，调试的时候，可以将上述的算法参数打印信息，以及AI/AO算法前后生成的数据倒出来，放到电脑上用audacity工具参看，标出时间轴，定位需要优化的地方。

10.2 固件大小

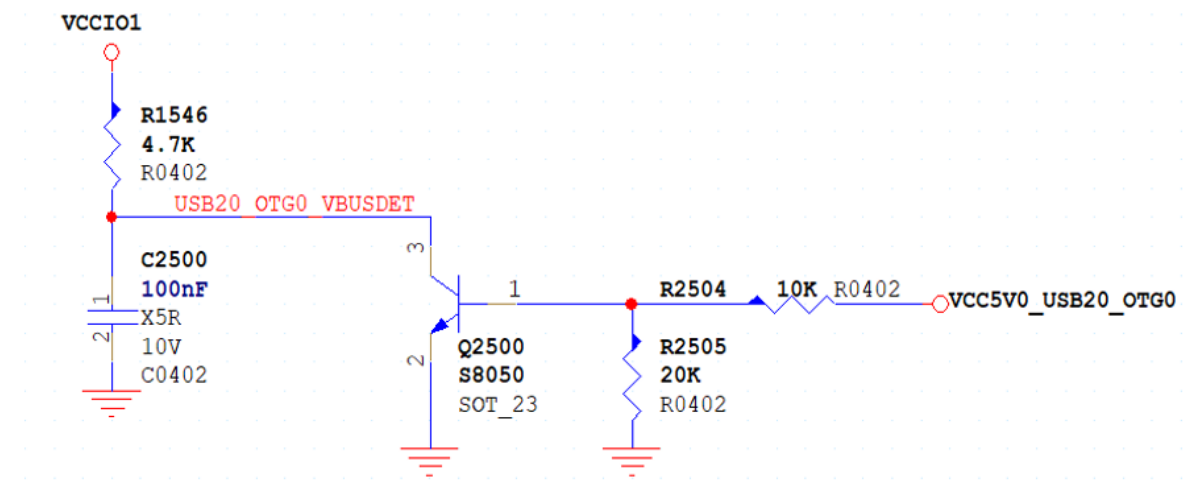
固件大小的裁剪，可以参考文档章节8.5中选择不同的配置组合来得到相应大小的固件，可以从U-Boot，Kernel，Buildroot三个部分来进行裁剪配置。

10.3 OTG-ID 和 OTG-VBUS-DET 配置说明

RK3506G的usbphy 精简掉了 OTG-ID 和 OTG-VBUS 脚，可以用 GPIO 来模拟实现相关功能。

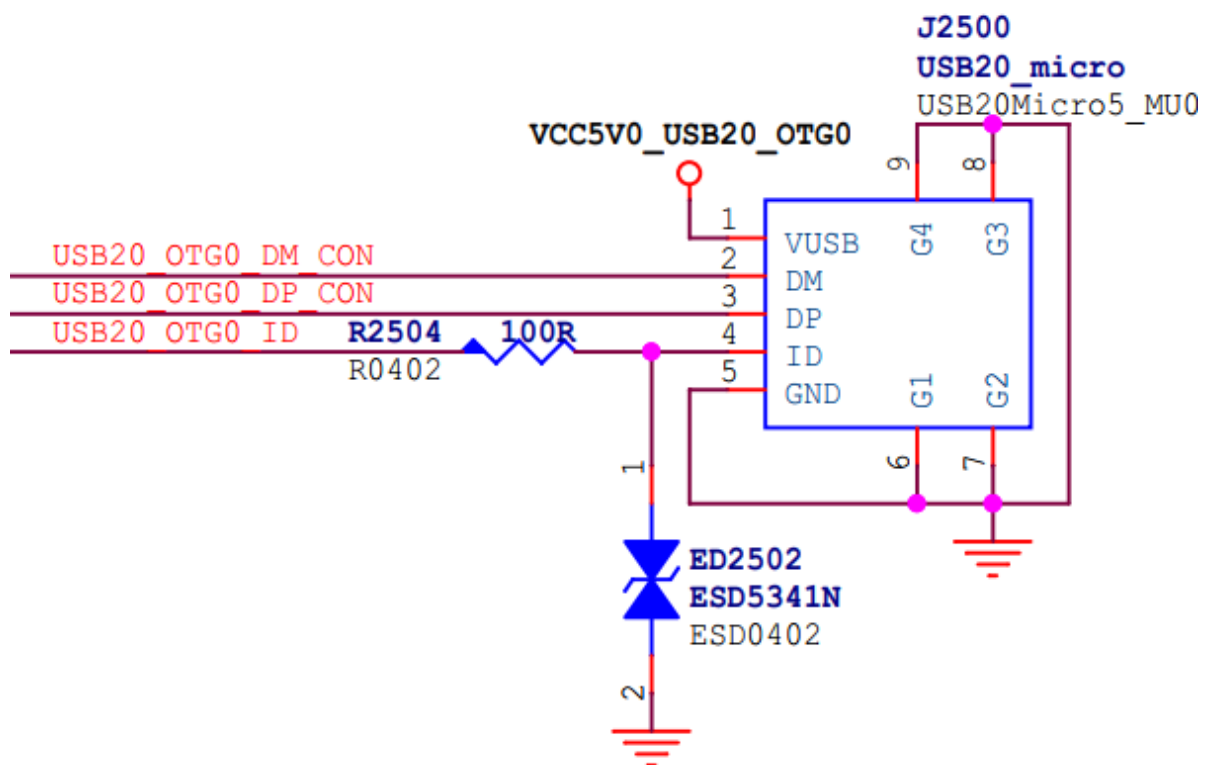
10.3.1 OTG ID 配置

OTG-ID 是用来实现 USB PERIPHERAL 和 USB HOST 模式自动切换功能。



USB20_OTG0_ID 接 GPIO，GPIO需要内部上拉。

OTG-VBUS-DET 是用来实现 USB DEVICE 拔插检测功能。



USB20_OTG0_VBUSDET 接 GPIO，VCCIO1 为 GPIO 电源域的供电电源。

VCC_5V0_USB20_OTG0 接 USB 母座的电源。

以上两个功能可以叠加使用。

DTS 配置如下：

```
/ {
    extcon_usb: extcon-usb {
        compatible = "linux,extcon-usb-gpio";
        vbus-gpio = <&gpio3 RK_PB2 GPIO_ACTIVE_LOW>; /* 定义 vbus-det-pin */
        id-gpio = <&gpio3 RK_PB4 GPIO_ACTIVE_HIGH>; /* 定义 id-det-pin */
    }
}
```



```

        pinctrl-names = "default";
        pinctrl-0 = <&usb_extcon_vbus &usb_extcon_id>; /* 引用pin的iomux */
        status = "okay";
    };

};

&u2phy_otg0 {
    vbus-supply = <&vcc5v0_otg0>;
    rockchip,gpio-id-det; /* 接受 id-det 通知 */
    rockchip,gpio-vbus-det; /* 接受 vbus-det 通知 */
    status = "okay";
};

&usb2phy {
    extcon = <&extcon_usb>; /* 引用 extcon_usb 的通知 */
    */
    status = "okay";
};

&pinctrl {
    usb {
        usb_extcon_id: usb-extcon-id {
            rockchip,pins = <3 RK_PB4 RK_FUNC_GPIO &pcfg_pull_up>; /* id 脚必须
上拉 */
        };

        usb_extcon_vbus: usb-extcon-vbus {
            rockchip,pins = <3 RK_PB2 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};

```

10.4 视频软解

视频软解使用CPU进行解码处理，多媒体VDEC模块已针对芯片类型自动选择硬件解码或软件解码，相关接口保持一致，用户无需感知。SDK中的RKADK播放器提供了相应的VDEC的创建与调用方式。

RKADK API参考文档路径如下：

```
<SDK>/docs/cn/Linux/Multimedia/Rockchip_Developer_Guide_Linux_RKADK_CN.pdf
```

目前视频软解仅支持H264解码，并且视频软解支持输出以下图像格式：

```

RK_FMT_YUV420P
RK_FMT_YUV420SP

```

视频软解的解码性能：

测试场景	帧率
CPU频率: 1.3G, 分辨率: 1280x720, 码率: 2862 kb/s (1 refs)	26
CPU频率: 1.3G, 分辨率: 720x512, 码率: 1136 kb/s (1 refs)	59
CPU频率: 1.5G, 分辨率: 1280x720, 码率: 2862 kb/s (1 refs)	30
CPU频率: 1.5G, 分辨率: 720x512, 码率: 1136 kb/s (1 refs)	72

可以由下面命令进行测试:

```
export send_data_debug_level=1
rkadk_player_test -i "/oem/720x512.mp4" -v -a 1 -s 0 -I 1 -P 0
```

10.5 MCU

RK3506 的 MCU 目前只提供裸核 Bare-Metal 的支持, SDK 全自动编译使用 its 配置文件来指定编译的 CPU 类型 (AP/MCU), 如修改 device/rockchip/rk3506/amp_linux.its, 添加 MCU 节点

```
/dts-v1/;
/ {
    description = "FIT source file for rockchip AMP";
    #address-cells = <1>;

    images {
        amp2 {
            description = "rtos-core2";
            data = /incbin/("rtt2.bin");
            type = "firmware";
            compression = "none";
            arch = "arm"; // "arm64" or "arm"
            cpu = <0xf02>; // mpidr
            thumb = <0>; // 0: arm or thumb2; 1: thumb
            hyp = <0>; // 0: el1/svc; 1: el2/hyp
            load = <0x03e00000>;
            srambase = <0xffff80000>;
            sramsize = <0x0000c000>;
            compile {
                size = <0x00100000>;
                sys = "rtt";
                core = "ap";
                rtt_config = "board/evb1/defconfig";
            };
            udelay = <10000>;
            hash {
                algo = "sha256";
            };
        };

        mcu {
            description = "mcu";
            data = /incbin/("mcu.bin");
            type = "standalone"; # must be "standalone"
```



```

        compression = "none";
        arch        = "arm";           # "arm64" or "arm", the same as U-
Boot state
        load        = <0xffff84000>;
        udelay      = <1000000>;
        compile {
            size      = <0x00800000>;
            sys       = "hal";
            core      = "mcu";         # 处理器核心类型: ap or mcu
        };
        hash {
            algo = "sha256";
        };
    };

    configurations {
        default = "conf";
        conf {
            description = "Rockchip AMP images";
            rollback-index = <0x0>;
            loadables = "amp2", "mcu";

            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "loadables";
            };

            /* - run linux on cpu0
             * - it is brought up by amp(that run on U-Boot)
             * - it is boot entry depends on U-Boot
             */
            linux {
                description = "linux-os";
                arch        = "arm";
                cpu         = <0xf00>;
                thumb       = <0>;
                hyp         = <0>;
                load        = <0x00900000>;
                udelay      = <0>;
            };
        };
    };
};

```

its 中添加需要编译的 mcu 节点后，defconfig 中指定编译使用的 its 的文件配置：

```
RK_AMP_FIT_ITS="amp_linux.its"
```

也可以进入 hal 下单独编译 MCU 固件：

```
cd hal/project/rk3506-mcu/GCC
make clean;make -j16
cd ..
./mkimage.sh
ls Image/amp.img -l
-rw-rw-r-- 1 zjh zjh 28672 Aug 22 09:31 Image/amp.img
```

MCU 的配置使用参考AMP开发文档：

SDK/docs/cn/Common/AMP/Rockchip_Developer_Guide_AMP_CN.pdf

10.6 Flexbus(DVP、高速IO)

flexbus功能请参考：

SDK/docs/cn/Common/FLEXBUS

10.7 RS485硬件方案自动切换

RS485硬件自动切换方案，参考文档添加属性：

```
linux, RS485-enabled -at-boot-time;
```

可查阅文档：

<SDK>/kernel/Documentation/devicetree/bindings/serial/rs485.yaml

10.8 实时性优化

隔离CPU1核心，并在另外两个核心加上压力：

```
stress-ng -c 3 --io 2 --vm 1 --vm-bytes 4M --timeout 1000000s
```

关掉显示

```
killall rk_demo
```

PREEMPT_RT

```
root@rk3506-buildroot:/# cyclicttest -m -a 1 -p 99 -t 1 -i 1000 --mainaffinity=0 -
D 2h
# /dev/cpu_dma_latency set to 0us
policy: fifo: loadavg: 7.19 7.17 7.22 7/137 648

T: 0 ( 1117) P:99 I:1000 C:7199996 Min:      0 Act:      2 Avg:      3 Max:      62
(测试2小时)
```

Xenomai Cobalt Mode

```
root@rk3506-buildroot:/# cyclicttest -m -a 1 -p 99 -t 1 -i 1000 -D 2h
WARN: stat /dev/cpu_dma_latency failed: No such file or directory
policy: fifo: loadavg: 6.53 6.52 6.57 8/101 1115

T: 0 ( 1503) P:99 I:1000 C:7199992 Min:      4 Act:   15 Avg:   16 Max:      68
(测试2小时)
```

实时性补丁和介绍文档:

```
docs/Patches/Real-Time-Performance/
├─ PREEMPT_RT
├─ Rockchip_Develop_Guide_Linux_RealTime_CN.md
├─ Rockchip_Develop_Guide_Linux_RealTime_Performance_Test_Report_CN.pdf
└─ XENOMAI
```

10.9 EtherCat

EtherCat文档请参考:

```
<SDK>/docs/cn/Linux/ApplicationNote/Rockchip_Use_Guide_Linux_EtherCAT_IgH_CN.pdf
```

10.10 CAN

CAN文档请参考:

```
<SDK>/docs/cn/Common/CAN/Rockchip_Developer_Guide_CAN_FD_CN.pdf
<SDK>/docs/cn/Common/CAN/Rockchip_Developer_Guide_Can_CN.pdf
```

10.11 DSMC

DSMC文档请参考:

```
<SDK>/docs/cn/Common/DSMC
```

10.12 LVGL Demo

目前RK3506默认使用RK_DEMO, 包含智能家居、家电显控、楼宇对讲、系统设置内容。在默认配置rockchip_rk3506_defconfig中已包含相关配置项:

```
...  
#include "lvgl/lvgl_rkadk.config"  
#include "lvgl/rk_demo.config"  
...
```

其余LVGL相关配置项可参考

<SDK>/docs/cn/Linux/Graphics/Rockchip_Developer_Guide_Linux_LVGL_CN.pdf 了解。

10.12.1 功能开启

进入buildroot menuconfig菜单，即可搜索并开关以下功能：

```
# 开启多媒体功能支持，本地视频播放、RTSP播放、语音对讲等  
BR2_RK_DEMO_ENABLE_MULTIMEDIA  
# 开启传感器功能支持  
BR2_RK_DEMO_ENABLE_SENSOR  
# 开启WiFi、蓝牙功能支持  
BR2_RK_DEMO_ENABLE_WIFIBT  
# 开启ASR功能支持  
BR2_RK_DEMO_ENABLE_ASR
```

10.13 显示部分

RK3506 支持 RGB、MIPI DSI 等显示接口，显示部分文档请参考：

<SDK>/docs/cn/Common/DISPLAY/

10.14 可读写文件系统

目前RK3506支持多种文件系统，大部分产品都是使用小内存，rootfs默认打包出来的都是ubi_squashfs类型，rootfs为只读的，这样可以减少使用空间，降低成本，如果需要读写的文件系统，需要修改rootfs的打包方式，如下：

内核修改rootfs类型：

```
diff --git a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi b/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
index 77f19b4c05c6..e2fe2f4c7c83 100644
--- a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
@@ -12,7 +12,7 @@ / {
    compatible = "rockchip,rk3506-evb1-v10", "rockchip,rk3506";

    chosen {
-       bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
ubi.mtd=5 ubi.block=0,rootfs root=/dev/ubiblock0_0 rootfstype=squashfs rootwait
snd_aloop.index=7 snd_aloop.use_raw_jiffies=1";
+       bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
ubi.mtd=5 root=ubi0:rootfs rw rootfstype=ubifs rootwait snd_aloop.index=7
snd_aloop.use_raw_jiffies=1";
    };

    acodec_sound: acodec-sound {
```

buildroot修改配置文件:

```
diff --git a/configs/rockchip_rk3506_defconfig
b/configs/rockchip_rk3506_defconfig
index 0908a3e48f..cd5e1786b2 100644
--- a/configs/rockchip_rk3506_defconfig
+++ b/configs/rockchip_rk3506_defconfig
@@ -8,6 +8,7 @@
 #include "wifibt/wireless.config"
 #include "lvgl/lvgl_rk3506.config"
 #include "lvgl/rk_demo.config"
+#include "fs/ubifs.config"
 # BR2_PACKAGE_CHRONY is reset to default
 # BR2_PACKAGE_DNSMASQ is reset to default
 # BR2_PACKAGE_DROPBEAR is reset to default
@@ -24,7 +25,7 @@ BR2_ROCKCHIP_ALSA_CONFIG_INSTALL_INIT_SCRIPT=y
 # BR2_TARGET_GENERIC_REMOUNT_ROOTFS_RW is not set
 BR2_TARGET_ROOTFS_SQUASHFS4_ZSTD=y
 BR2_TARGET_ROOTFS_UBI=y
-BR2_TARGET_ROOTFS_UBI_SQUASHFS=y
+BR2_TARGET_ROOTFS_UBI_UBIFS=y
 # BR2_TARGET_ROOTFS_UBI_SQUASHFS_STATIC is not set
 BR2_TARGET_ROOTFS_UBI_SUBSIZE=2048
 BR2_TIME_BITS_64=y
```

注意，改为可读写文件系统后，文件系统镜像会变大，需要对应的调整分区表。

10.15 使用eMMC存储介质

SDK中默认所有的存储介质为SPI NAND，如果想使用eMMC存储介质，需要做如下修改:

SDK/device/rockchip/rk3506/parameter-128M.txt

```

diff --git a/.chips/rk3506/parameter-128M.txt b/.chips/rk3506/parameter-128M.txt
index 8147cd5..3c2ba99 100644
--- a/.chips/rk3506/parameter-128M.txt
+++ b/.chips/rk3506/parameter-128M.txt
@@ -9,5 +9,5 @@ CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0
-
CMDLINE:mtdparts=:0x00002000@0x00002000 (uboot),0x00000800@0x00004000 (misc),0x0000
0200@0x00004800 (vnvm),0x00007000@0x00004a00 (recovery),0x00005000@0x0000ba00 (boot)
,0x00020000@0x00010a00 (rootfs),0x00008000@0x00030a00 (oem),-
@0x00038a00 (userdata:grow)
+CMDLINE:mtdparts=:0x00002000@0x00004000 (uboot),0x00002000@0x00006000 (misc),0x000
20000@0x00008000 (boot),0x00040000@0x00028000 (recovery),0x00010000@0x00068000 (back
up),0x00c00000@0x00078000 (rootfs),0x00040000@0x00c78000 (oem),-
@0x00cb8000 (userdata:grow)
uuid:rootfs=614e0000-0000-4b53-8000-1d28000054a9
diff --git a/.chips/rk3506/rockchip_rk3506_b_evb1_defconfig
b/.chips/rk3506/rockchip_rk3506_b_evb1_defconfig
index e1b6d3f..dc000cf 100644
--- a/.chips/rk3506/rockchip_rk3506_b_evb1_defconfig
+++ b/.chips/rk3506/rockchip_rk3506_b_evb1_defconfig
@@ -1,20 +1,17 @@
-RK_ROOTFS_UBI=y
RK_ROOTFS_INSTALL_MODULES=y
RK_WIFIBT_CHIP="AP6256"
# RK_ROOTFS_LOG_GUARDIAN is not set
RK_UBOOT_CFG="rk3506"
-RK_UBOOT_CFG_FRAGMENTS="rk3506b rk3506_tb"
+RK_UBOOT_CFG_FRAGMENTS="rk3506b rk-emmc"
RK_UBOOT_SPL=y
RK_KERNEL_ARM32=y
RK_KERNEL_CFG="rk3506_defconfig"
RK_KERNEL_CFG_FRAGMENTS="rk3506-display.config"
RK_KERNEL_DTS_NAME="rk3506b-evb1-v10"
RK_BOOT_COMPRESSED=y
-RK_BOOT_FIT_ITS_NAME="thunderboot.its"
RK_RECOVERY_FIT_ITS_NAME="zboot4recovery.its"
-RK_FLASH_SIZE=2048
-RK_EXTRA_PARTITION_1_FSTYPE="ubi"
+RK_EXTRA_PARTITION_1_FSTYPE="ext2"
RK_EXTRA_PARTITION_1_SRC="rk3506_oem"
-RK_EXTRA_PARTITION_2_FSTYPE="ubi"
+RK_EXTRA_PARTITION_2_FSTYPE="ext2"
RK_PARAMETER="parameter-128M.txt"
RK_USE_FIT_IMG=y

```

SDK/u-boot:

```

From aff710b40d23e8d2b8b76f16f7d967bf7d4af82e Mon Sep 17 00:00:00 2001
From: Xuhui Lin <xuhui.lin@rock-chips.com>
Date: Tue, 22 Oct 2024 14:17:44 +0800
Subject: [PATCH] rockchip: rk3506: Add unset mmc iomux support

```

Because rk3506 evb boards don't have sdmmc_det by default,
remove mmc support in rk3506_defconfig.

If need mmc support, please:

- (1) make rk3506_defconfig rk-emmc.config
- (2) ./make.sh --spl-new

Change-Id: I626f49ba069bd4739a441c7fa677cccb041280a8

Signed-off-by: Xuhui Lin <xuhui.lin@rock-chips.com>

```
arch/arm/dts/rk3506-u-boot.dtsi          |  2 +-
arch/arm/mach-rockchip/rk3506/rk3506.c    | 70 ++++++-----
configs/rk-emmc.config                    |  5 ++
configs/rk3506_defconfig                  |  5 +-
4 files changed, 60 insertions(+), 22 deletions(-)
```

diff --git a/arch/arm/dts/rk3506-u-boot.dtsi b/arch/arm/dts/rk3506-u-boot.dtsi

index 2b61487..7096f34 100644

--- a/arch/arm/dts/rk3506-u-boot.dtsi

+++ b/arch/arm/dts/rk3506-u-boot.dtsi

@@ -11,7 +11,7 @@

```
        chosen: chosen {
            stdout-path = &uart0;
-       u-boot,spl-boot-order = &spi_nand, &spi_nor;
+       u-boot,spl-boot-order = &mmc, &spi_nand, &spi_nor;
        };
    };
```

diff --git a/arch/arm/mach-rockchip/rk3506/rk3506.c b/arch/arm/mach-rockchip/rk3506/rk3506.c

index a147f54..cf919cf 100644

--- a/arch/arm/mach-rockchip/rk3506/rk3506.c

+++ b/arch/arm/mach-rockchip/rk3506/rk3506.c

@@ -5,6 +5,7 @@

```
    */
    #include <common.h>
    #include <dm.h>
+   #include <spl.h>
    #include <asm/io.h>
    #include <asm/arch/cpu.h>
    #include <asm/arch/hardware.h>
@@ -45,23 +46,6 @@ void board_debug_uart_init(void)
    /* No need to change uart*/
}

-#ifndef CONFIG_SPL_BUILD
-void rockchip_stimer_init(void)
-{
-    /* If Timer already enabled, don't re-init it */
-    u32 reg = readl(CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
-    if (reg & 0x1)
-        return;
-    writel(0x00010000, CONFIG_ROCKCHIP_STIMER_BASE + 0x4);
-}
-#endif
```



```

+   if (!loader)
+       return;
+
+   if (loader->boot_device == BOOT_DEVICE_MMC1 ||
+       loader->boot_device == BOOT_DEVICE_MMC2)
+       /* Unset the sdmmc0 iomux */
+       board_unset_iomux(IF_TYPE_MMC, 0, 0);
+}
+
+
+int arch_cpu_init(void)
+{
+    #if defined(CONFIG_SPL_BUILD)
@@ -92,9 +125,10 @@ int arch_cpu_init(void)
+        val = readl(FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
+        writel(val & 0xffff00ff, FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
+
+
+    #if defined(CONFIG_MMC_DW_ROCKCHIP)
+        /* Set the sdmmc/emmc iomux */
+        board_set_iomux(IF_TYPE_MMC, 0, 0);
+
+    -
+
+    #endif
+
+    /* Set the fspi to access ddr memory */
+    val = readl(FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
+    writel(val & 0xff00ffff, FIREWALL_DDR_BASE + FW_DDR_MST1_REG);
diff --git a/configs/rk-emmc.config b/configs/rk-emmc.config
index 5e7dad3..bf9548b 100644
--- a/configs/rk-emmc.config
+++ b/configs/rk-emmc.config
@@ -1,2 +1,7 @@
+CONFIG_ROCKCHIP_EMMC_IOMUX=y
+
+# CONFIG_ROCKCHIP_SFC_IOMUX is not set
+CONFIG_CMD_MMC=y
+CONFIG_MMC=y
+CONFIG_DM_MMC=y
+CONFIG_MMC_DW=y
+CONFIG_MMC_DW_ROCKCHIP=y
diff --git a/configs/rk3506_defconfig b/configs/rk3506_defconfig
index c473a55..1ee0e96 100644
--- a/configs/rk3506_defconfig
+++ b/configs/rk3506_defconfig
@@ -57,7 +57,6 @@ CONFIG_RANDOM_UUID=y
+
+# CONFIG_CMD_LOADB is not set
+# CONFIG_CMD_LOADS is not set
+CONFIG_CMD_BOOT_ANDROID=y
-CONFIG_CMD_MMC=y
+CONFIG_CMD_MTD=y
+
+# CONFIG_CMD_ITEST is not set
+CONFIG_CMD_SCRIPT_UPDATE=y
@@ -100,8 +99,8 @@ CONFIG_MISC=y
+CONFIG_SPL_MISC=y
+CONFIG_ROCKCHIP_OTP=y
+
+# CONFIG_SUPPORT_EMMC_RPMB is not set
-CONFIG_MMC_DW=y
-CONFIG_MMC_DW_ROCKCHIP=y

```

```

+# CONFIG_MMC is not set
+# CONFIG_DM_MMC is not set
CONFIG_MTD=y
CONFIG_MTD_BLK=y
CONFIG_MTD_DEVICE=y
--
2.34.1

```

SDK/kernel:

```

diff --git a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi b/arch/arm/boot/dts/rk3502-
evb1-v10.dtsi
index 8f4e57790..4795c7d0a 100644
--- a/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3502-evb1-v10.dtsi
@@ -194,6 +194,16 @@ vcc3v3_stb: vcc3v3-stb {
    vin-supply = <&vcc_sys>;
};

+   vcc_3v3: vcc-3v3 {
+       compatible = "regulator-fixed";
+       regulator-name = "vcc_3v3";
+       regulator-always-on;
+       regulator-boot-on;
+       regulator-min-microvolt = <3300000>;
+       regulator-max-microvolt = <3300000>;
+       vin-supply = <&vcc_sys>;
+   };
+
+   vcc_1v8: vcc-1v8 {
+       compatible = "regulator-fixed";
+       regulator-name = "vcc_1v8";
@@ -321,6 +331,7 @@ es8388: es8388@11 {
};

&mmc {
+   /*
+       bus-width = <4>;
+       cap-sd-highspeed;
+       no-sd;
@@ -332,6 +343,20 @@ &mmc {
+       non-removable;
+       mmc-pwrseq = <&sdio_pwrseq>;
+       sd-uhs-sdr104;
+   */
+
+   //For eMMC
+   no-sdio;
+   no-sd;
+   cap-mmc-highspeed;
+   cap-sd-highspeed;
+   non-removable;
+   mmc-hs200-1_8v;
+   pinctrl-names = "default";
+   pinctrl-0 = <&sdmmc_clk_pins &sdmmc_cmd_pins &sdmmc_bus4_pins>;

```

```

+ //vqmmc-supply = <&vccio4>;
+ //vmmc-supply = <&vcc_3v3>;
+
+     status = "okay";
+ };

diff --git a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi b/arch/arm/boot/dts/rk3506-
evb1-v10.dtsi
index b1b182de7..1e0a47171 100644
--- a/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
+++ b/arch/arm/boot/dts/rk3506-evb1-v10.dtsi
@@ -13,7 +13,7 @@ / {
     compatible = "rockchip,rk3506-evb1-v10", "rockchip,rk3506";

     chosen {
-        bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0 ubi.mtd=5
ubi.block=0,rootfs root=/dev/ubiblock0_0 rootfstype=squashfs rootwait
snd_aloop.index=7 snd_aloop.use_raw_jiffies=1";
+        bootargs = "earlycon=uart8250,mmio32,0xff0a0000 console=ttyFIQ0
root=PARTUUID=614e0000-0000 rootwait snd_aloop.index=7
snd_aloop.use_raw_jiffies=1";
     };

     backlight: backlight {
diff --git a/arch/arm/configs/rk3506_defconfig
b/arch/arm/configs/rk3506_defconfig
index d6cfc7e6..39fd1cc65 100644
--- a/arch/arm/configs/rk3506_defconfig
+++ b/arch/arm/configs/rk3506_defconfig
@@ -253,6 +253,9 @@ CONFIG_USB_GADGET=m
CONFIG_USB_CONFIGFS=m
CONFIG_USB_CONFIGFS_UEVENT=y
CONFIG_USB_CONFIGFS_F_FS=y
+CONFIG_MMC=y
+CONFIG_MMC_DW=y
+CONFIG_MMC_DW_ROCKCHIP=y
CONFIG_USB_ROLE_SWITCH=y
CONFIG_MMC=y
CONFIG_MMC_QUEUE_DEPTH=1
@@ -303,7 +306,8 @@ CONFIG_PWM_ROCKCHIP=y
CONFIG_PHY_ROCKCHIP_INNO_USB2=m
CONFIG_PHY_ROCKCHIP_INNO_DSIDPHY=y
CONFIG_NVMEM_ROCKCHIP_OTP=y
-CONFIG_EXT4_FS=m
+CONFIG_EXT2_FS=y
+CONFIG_EXT4_FS=y
# CONFIG_DNOTIFY is not set
CONFIG_VFAT_FS=m
CONFIG_EXFAT_FS=m

```